

The subnet mask is 255.255.255.128; therefore, the possible IP addresses are from 192.168.100.1 to 192.168.100.126. Since there are just two nodes, the addresses are 192.168.100.1 and 192.168.100.2.

During our discussion on ns-3 abstractions, I mentioned abstract applications running on ns-3 nodes. Now it is time for us to set up abstract applications on top of the two nodes. The lines from `'UdpEchoServerHelper echoServer(2000);'` to `'clientApps.Stop(Seconds(12.0));'` in the script `simulation1.cc` are used to create abstract applications on the two nodes. The node with address 192.168.100.1 is designated as the client, and the node with address 192.168.100.2 is designated as the server.

Initially, we create an echo server application on the server node. The port used by the echo server is 2000. The line `'ApplicationContainer serverApps = echoServer.Install(nodes.Get(1));'` installs a UDP echo server application on the node with ID 1. The server application starts at the first second and stops at the tenth second. Then we create a UDP echo client on the node with ID 0, and it connects to the server with the line `'UdpEchoClientHelper echoClient(interfaces.GetAddress(1), 2000);'`. Remember the connection mechanisms we learned in socket programming? With the next three lines of code, the maximum number of packets sent is set as two, the interval is set as 3 seconds and the packet size is fixed as 512 bytes. The client application starts at the third second and stops at the twelfth second. So with the simulation script given above we have created nodes, installed applications on the nodes, attached network devices on the nodes, and connected the nodes to a communication channel. Thus our topology is complete and we are ready to begin the simulation. And finally, the lines `'Simulator::Run();'` and `'Simulator::Destroy();'` start and stop the actual ns-3 simulator.

Trace files in ns-3

I have explained every line of code in the script `simulation1.cc` except the following two lines:

```
AsciiTraceHelper ascii;
pointToPoint.EnableAsciiAll(ascii);
CreateFileStream("simulation1.tr");
```

These two lines are included in the script for trace file generation. Trace file analysis is the main source of information in ns-3 simulation. There are two different types of trace files in ns-3. They are ASCII based trace files and *pcap* based trace files. *pcap* is an application programming interface (API) for capturing network traffic. A packet analyser tool called Wireshark is used to process the *pcap* files to elicit information. Initially, we will only discuss the relatively simple ASCII based tracing method. The code given above will generate an ASCII trace file named `simulation1.tr`. The text below shows only the first line of the trace file.

```
+ 3 /NodeList/0/DeviceList/0/$ns3::PointToPointNetDevice/
TxQueue/Enqueue ns3::PppHeader (Point-to-Point Protocol: IP
(0x0021)) ns3::Ipv4Header (tos 0x0 DSCP Default ECN Not-ECT
ttl 64 id 0 protocol 17 offset (bytes) 0 flags [none] length:
540 192.168.100.1 > 192.168.100.2) ns3::UdpHeader (length:
520 49153 > 2000) Payload (size=512)
```

You can download the simulation script `simulation1.cc` and the corresponding trace file `simulation1.tr` from opendsa.org.uk/efytimes.com/article_source_code/july2015/ns3.zip. The ASCII trace files in ns-3 are similar in structure to ns-2 trace files. There is abundant data available in the form of a trace file in ns-3. A detailed description of the different fields in the trace file and ways to elicit information from this file will be discussed in the next issue.

Executing the script

Now that we are familiar with the code, let's move on to the execution of the code. If you have installed ns-3 inside a directory called *ns* as discussed in the last article of this series, you will have the following directory `ns/ns-allinone-3.22/ns-3.22/scratch`. Copy the file `simulation1.cc` into the directory called *scratch*. You have to copy ns-3 scripts into this directory so that the build automation tool, *Waf*, can identify and compile them. Now open a terminal in the directory `ns/ns-allinone-3.22` and type the following command:

```
./waf
```

If your script is incorrect, you will see error messages displayed on the terminal. If the simulation script is correct, you will see a message regarding a list of modules built and a few modules not built. If you see such a message, then type the following command in the terminal:

```
./waf - - run scratch/simulation1
```

This command will execute the simulation script `simulation1.cc`. Figure 1 shows the terminal with logged information displayed. From the figure, we can see that the client sends 512 bytes of data to the server at the third second. The server responds with 512 bytes of data to the client. After a waiting period of 3 seconds, at the sixth second, the client again sends 512 bytes to the server and the server in turn responds by sending 512 bytes of data to the client.

Graphical animation in ns-3

We have briefly discussed the NAM graphical animator used with ns-2 in the first part of this series. We are now going to deal with network animation in ns-3. Unlike ns-2, there is no single default animator for ns-3. But we have two tools called *NetAnim* and *PyViz* for network animation in ns-3. *PyViz* is a live simulation visualiser, which means it doesn't use trace files. *NetAnim* is a trace based network animator. I will